

Meta Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **"Move fast"** culture — shipping speed valued over perfection
- Data-intensive by nature: billions of users, petabytes of daily data
- Infrastructure at extreme scale: everything is distributed
- Internal culture of open source and open data tooling (Presto/Trino, RocksDB, PyTorch)
- Flat structure with high individual engineer impact
- Strong focus on infrastructure reliability (SRE culture embedded in eng teams)

Database Technology Stack

Layer	Technology
Social Graph	TAO (custom graph store, built on MySQL/RocksDB)
News Feed	RocksDB-based custom stores
OLAP / Analytics	Presto (now Trino), Hive, Spark
Data Lake	HDFS → Data Lakehouse (Iceberg format)
OLTP	MySQL (custom distributed), PostgreSQL (internal tools)
Caching	Memcache (Mcrouter), Tao cache
Stream Processing	Flink, Scribe (custom log infra)
Time Series	Gorilla (Meta's in-memory TSDB), ODS
Cold Storage	Meta's custom cold tier
ML Feature Store	Custom feature store on RocksDB

What Meta Values in DB/SQL Candidates

- **Scale-first thinking**: every design starts at billions of rows
 - Deep data engineering: understanding distributed joins, shuffles, partitioning
 - Practical performance: Presto/SQL optimization at petabyte scale
 - Systems thinking: how data flows end-to-end in infrastructure
 - Impact-oriented: can you ship something that affects billions of users?
 - Operational excellence: oncall culture, automation over manual operations
-

2. Interview Process

Typical Rounds (E4/E5/E6 Engineer, Data Engineer)

Round	Format	Duration	Focus
-------	--------	----------	-------

Recruiter Call	Phone	30 min	Background, interest
Technical Screen	CoderPad	60 min	SQL + coding
Onsite/Virtual 1	CoderPad	45 min	SQL + algorithms
Onsite/Virtual 2	Video	45 min	System design (data)
Onsite/Virtual 3	Video	45 min	Infrastructure design
Onsite/Virtual 4	Video	45 min	Behavioral
Onsite/Virtual 5	Video	45 min	Additional technical

Level Expectations

Level	Equivalent	SQL Expectation	Scale Thinking
E3	Junior	Medium SQL	Millions of rows
E4	Mid-level	Hard SQL + optimization	Hundreds of millions
E5	Senior	Expert + distributed SQL	Billions of rows
E6	Staff	Novel design + leadership	Petabyte-scale

3. SQL Coding Round — 25 Questions at Meta Scale

Q1. Find users who have friends in more than 3 different countries.

```
-- Meta has a friends graph; translated to SQL:
SELECT f.user_id, COUNT(DISTINCT u.country) AS friend_countries
FROM friendships f
JOIN users u ON f.friend_id = u.user_id
GROUP BY f.user_id
HAVING COUNT(DISTINCT u.country) > 3;
```

Q2. Calculate the virality coefficient: average number of new users each user invited who then invited others.

```
WITH direct_invites AS (
  SELECT referrer_id, COUNT(*) AS users_invited
  FROM referrals
  GROUP BY referrer_id
),
second_level AS (
  SELECT r1.referrer_id AS original_referrer, COUNT(r2.referred_user_id) AS
secondary_invites
  FROM referrals r1
  JOIN referrals r2 ON r1.referred_user_id = r2.referrer_id
  GROUP BY r1.referrer_id
)
```

```

SELECT AVG(COALESCE(sl.secondary_invites, 0) / NULLIF(di.users_invited, 0)) AS
virality_k
FROM direct_invites di
LEFT JOIN second_level sl ON di.referrer_id = sl.original_referrer;

```

Q3. Find all posts where engagement rate (reactions/impressions) is in the top 1%.

```

WITH post_engagement AS (
    SELECT
        post_id,
        total_reactions::numeric / NULLIF(total_impressions, 0) AS engagement_rate,
        PERCENT_RANK() OVER (ORDER BY total_reactions::numeric /
NULLIF(total_impressions, 0) DESC) AS pct_rank
    FROM post_metrics
    WHERE total_impressions >= 100 -- minimum viable impressions
)
SELECT post_id, engagement_rate
FROM post_engagement
WHERE pct_rank <= 0.01
ORDER BY engagement_rate DESC;

```

Q4. Detect bot accounts: users with > 1000 actions in any single hour.

```

SELECT
    user_id,
    DATE_TRUNC('hour', action_time) AS hour_bucket,
    COUNT(*) AS action_count
FROM user_actions
GROUP BY user_id, DATE_TRUNC('hour', action_time)
HAVING COUNT(*) > 1000
ORDER BY action_count DESC;

```

Q5. Friend suggestion: find users with 5+ mutual friends but who aren't connected.

```

WITH mutual_friends AS (
    SELECT
        f1.user_id AS user_a,
        f2.user_id AS user_b,
        COUNT(*) AS mutual_count
    FROM friendships f1
    JOIN friendships f2 ON f1.friend_id = f2.friend_id
                        AND f1.user_id <> f2.user_id
    GROUP BY f1.user_id, f2.user_id
    HAVING COUNT(*) >= 5
)
SELECT mf.user_a, mf.user_b, mf.mutual_count
FROM mutual_friends mf
WHERE NOT EXISTS (
    SELECT 1 FROM friendships f
    WHERE (f.user_id = mf.user_a AND f.friend_id = mf.user_b)
)

```

```

        OR (f.user_id = mf.user_b AND f.friend_id = mf.user_a)
    )
    ORDER BY mf.mutual_count DESC;

```

Q6. WhatsApp-scale: Find the top 10 most active group chats in the last 24 hours.

```

SELECT
    group_id,
    COUNT(*) AS message_count,
    COUNT(DISTINCT sender_id) AS active_members,
    MAX(sent_at) AS last_message
FROM messages
WHERE sent_at >= NOW() - INTERVAL '24 hours'
GROUP BY group_id
ORDER BY message_count DESC
LIMIT 10;

```

Q7. Instagram: Calculate story completion rate by account type (creator vs. personal).

```

SELECT
    u.account_type,
    COUNT(DISTINCT sv.story_id) AS total_story_views,
    AVG(sv.completion_pct) AS avg_completion_pct,
    COUNT(*) FILTER (WHERE sv.completion_pct = 100) AS full_views,
    COUNT(*) FILTER (WHERE sv.completion_pct < 50) AS early_exits
FROM story_views sv
JOIN stories s ON sv.story_id = s.story_id
JOIN users u ON s.author_id = u.user_id
GROUP BY u.account_type;

```

Q8. News Feed: Find posts that are trending — gaining reactions faster than average for their age.

```

WITH post_stats AS (
    SELECT
        p.post_id,
        p.created_at,
        EXTRACT(EPOCH FROM (NOW() - p.created_at)) / 3600 AS age_hours,
        COUNT(r.reaction_id) AS reaction_count,
        COUNT(r.reaction_id)::numeric /
            NULLIF(EXTRACT(EPOCH FROM (NOW() - p.created_at)) / 3600, 0) AS
reactions_per_hour
    FROM posts p
    LEFT JOIN reactions r ON p.post_id = r.post_id
    WHERE p.created_at >= NOW() - INTERVAL '48 hours'
    GROUP BY p.post_id, p.created_at
)
SELECT post_id, age_hours, reaction_count, reactions_per_hour
FROM post_stats

```

```
WHERE reactions_per_hour > (SELECT AVG(reactions_per_hour) * 2 FROM post_stats)
ORDER BY reactions_per_hour DESC;
```

Q9. Find the shortest social distance between two users (up to 3 degrees).

```
WITH RECURSIVE social_distance AS (
  SELECT user_id AS start, friend_id AS current, 1 AS degree,
         ARRAY[user_id, friend_id] AS path
  FROM friendships WHERE user_id = :user_a

  UNION ALL

  SELECT sd.start, f.friend_id, sd.degree + 1, sd.path || f.friend_id
  FROM social_distance sd
  JOIN friendships f ON sd.current = f.user_id
  WHERE f.friend_id <> ALL(sd.path)
         AND sd.degree < 3
         AND NOT EXISTS (SELECT 1 FROM social_distance sd2
                          WHERE sd2.current = f.friend_id AND sd2.degree <= sd.degree)
)
SELECT path, degree
FROM social_distance
WHERE current = :user_b
ORDER BY degree
LIMIT 1;
```

Q10. A/B Test Analysis: calculate statistical significance of conversion rate difference.

```
SELECT
  variant,
  SUM(converted) AS conversions,
  COUNT(*) AS impressions,
  SUM(converted)::numeric / COUNT(*) AS cvr,
  -- Z-score for two proportions
  (SUM(converted)::numeric / COUNT(*) -
   SUM(SUM(converted)) OVER () / SUM(COUNT(*)) OVER ())
  / SQRT(
    (SUM(SUM(converted)) OVER () / SUM(COUNT(*)) OVER ()) *
    (1 - SUM(SUM(converted)) OVER () / SUM(COUNT(*)) OVER ()) /
    COUNT(*)
  ) AS z_score
FROM ab_test_assignments
WHERE test_id = :test_id
GROUP BY variant;
```

Q11. Ads: Calculate ROAS (Return on Ad Spend) with attribution window.

```
SELECT
  c.campaign_id,
  c.campaign_name,
```

```

SUM(a.ad_spend) AS total_spend,
SUM(o.order_value) AS attributed_revenue,
ROUND(SUM(o.order_value) / NULLIF(SUM(a.ad_spend), 0), 2) AS roas
FROM campaigns c
JOIN ad_spend_daily a ON c.campaign_id = a.campaign_id
LEFT JOIN conversions cv ON cv.campaign_id = c.campaign_id
                        AND cv.converted_at BETWEEN a.spend_date
                                                AND a.spend_date + INTERVAL '7
days'
LEFT JOIN orders o ON cv.order_id = o.order_id
GROUP BY c.campaign_id, c.campaign_name
ORDER BY roas DESC;

```

Q12. Feed ranking: combine recency and engagement score.

```

SELECT
  post_id,
  author_id,
  created_at,
  reaction_count,
  comment_count,
  share_count,
  -- Hacker News-style ranking: score / time_decay
  (reaction_count + comment_count * 2 + share_count * 3)::numeric /
    POWER(EXTRACT(EPOCH FROM (NOW() - created_at)) / 3600 + 2, 1.8) AS
feed_score
FROM posts
WHERE created_at >= NOW() - INTERVAL '7 days'
ORDER BY feed_score DESC
LIMIT 50;

```

Q13. Find all users who saw an ad but didn't click, and clicked but didn't convert.

```

SELECT
  'saw_no_click' AS stage,
  COUNT(DISTINCT user_id) AS users
FROM ad_impressions
WHERE NOT EXISTS (SELECT 1 FROM ad_clicks c WHERE c.user_id = ad_impressions.user_id
                  AND c.ad_id = ad_impressions.ad_id)

UNION ALL

SELECT
  'clicked_no_convert' AS stage,
  COUNT(DISTINCT user_id) AS users
FROM ad_clicks
WHERE NOT EXISTS (SELECT 1 FROM conversions cv WHERE cv.user_id = ad_clicks.user_id
                  AND cv.ad_id = ad_clicks.ad_id);

```

Q14. Instagram: Find accounts showing unusual follower growth (potential manipulation).

```

WITH daily_growth AS (
    SELECT
        account_id,
        snapshot_date,
        follower_count,
        follower_count - LAG(follower_count) OVER (PARTITION BY account_id ORDER BY
snapshot_date) AS daily_gain,
        AVG(follower_count - LAG(follower_count) OVER (PARTITION BY account_id ORDER
BY snapshot_date))
            OVER (PARTITION BY account_id ORDER BY snapshot_date ROWS BETWEEN 29
PRECEDING AND 1 PRECEDING) AS avg_30d_gain
    FROM follower_snapshots
)
SELECT account_id, snapshot_date, daily_gain, avg_30d_gain,
    ROUND(daily_gain / NULLIF(avg_30d_gain, 0), 1) AS growth_multiplier
FROM daily_growth
WHERE daily_gain > avg_30d_gain * 10
    AND daily_gain > 1000
ORDER BY growth_multiplier DESC;

```

Q15. Calculate Day N retention for a cohort of users.

```

WITH cohort AS (
    SELECT user_id, DATE(first_seen) AS cohort_date
    FROM (SELECT user_id, MIN(event_time) AS first_seen FROM events GROUP BY
user_id) t
),
retention AS (
    SELECT
        c.cohort_date,
        (DATE(e.event_time) - c.cohort_date) AS day_number,
        COUNT(DISTINCT c.user_id) AS retained_users
    FROM cohort c
    JOIN events e ON c.user_id = e.user_id
    WHERE (DATE(e.event_time) - c.cohort_date) IN (0, 1, 7, 14, 30)
    GROUP BY c.cohort_date, day_number
),
cohort_sizes AS (
    SELECT cohort_date, COUNT(*) AS cohort_size FROM cohort GROUP BY cohort_date
)
SELECT
    r.cohort_date,
    cs.cohort_size,
    r.day_number,
    r.retained_users,
    ROUND(100.0 * r.retained_users / cs.cohort_size, 1) AS retention_rate
FROM retention r
JOIN cohort_sizes cs USING (cohort_date)
ORDER BY cohort_date, day_number;

```

Q16. Presto/Trino-style: approximate distinct count with HLL.

```
-- In Presto (Meta's internal analytics):
SELECT
    DATE_TRUNC('day', event_time) AS day,
    APPROX_DISTINCT(user_id) AS approx_dau, -- HyperLogLog, ~2% error
    COUNT(*) AS total_events
FROM events
GROUP BY DATE_TRUNC('day', event_time);

-- PostgreSQL equivalent (requires pg_hll extension):
SELECT
    DATE(event_time) AS day,
    HLL_CARDINALITY(HLL_ADD_AGG(HLL_HASH_BIGINT(user_id))) AS approx_dau
FROM events
GROUP BY DATE(event_time);
```

Q17. Find content that violated policies multiple times across different platforms.

```
SELECT
    c.content_id,
    c.content_type,
    c.author_id,
    COUNT(DISTINCT v.platform) AS platforms_violated,
    COUNT(*) AS total_violations,
    ARRAY_AGG(DISTINCT v.violation_type) AS violation_types
FROM content c
JOIN policy_violations v ON c.content_id = v.content_id
GROUP BY c.content_id, c.content_type, c.author_id
HAVING COUNT(DISTINCT v.platform) >= 2
ORDER BY total_violations DESC;
```

Q18. Calculate P(engaged | saw post) for different content types.

```
SELECT
    p.content_type,
    COUNT(*) AS impressions,
    COUNT(*) FILTER (WHERE e.event_id IS NOT NULL) AS engagements,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE e.event_id IS NOT NULL) / COUNT(*), 4
    ) AS engagement_probability
FROM post_impressions pi
JOIN posts p ON pi.post_id = p.post_id
LEFT JOIN engagement_events e ON pi.post_id = e.post_id AND pi.user_id = e.user_id
GROUP BY p.content_type
ORDER BY engagement_probability DESC;
```

Q19. Ads bidding: find advertisers who consistently underbid and lose auctions.

```

WITH auction_results AS (
    SELECT
        advertiser_id,
        auction_id,
        bid_amount,
        winning_bid,
        bid_amount < winning_bid AS lost,
        (winning_bid - bid_amount) / NULLIF(winning_bid, 0) AS bid_gap_pct
    FROM auction_history
    WHERE occurred_at >= NOW() - INTERVAL '30 days'
)
SELECT
    advertiser_id,
    COUNT(*) AS total_auctions,
    SUM(lost::int) AS auctions_lost,
    ROUND(100.0 * SUM(lost::int) / COUNT(*), 2) AS loss_rate_pct,
    ROUND(AVG(bid_gap_pct) FILTER (WHERE lost), 4) AS avg_bid_gap_when_losing
FROM auction_results
GROUP BY advertiser_id
HAVING SUM(lost::int) > 100 AND ROUND(100.0 * SUM(lost::int) / COUNT(*), 2) > 70
ORDER BY avg_bid_gap_when_losing DESC;

```

Q20. Graph centrality: find users with highest betweenness (most connections bridging communities).

```

-- Simplified: degree centrality (full betweenness requires graph algorithms)
SELECT
    user_id,
    COUNT(*) AS degree,
    COUNT(*) FILTER (WHERE friendship_type = 'cross_community') AS
bridge_connections,
    ROUND(
        COUNT(*) FILTER (WHERE friendship_type = 'cross_community')::numeric /
        NULLIF(COUNT(*), 0), 4
    ) AS bridge_ratio
FROM friendships f
JOIN community_memberships cm1 ON f.user_id = cm1.user_id
JOIN community_memberships cm2 ON f.friend_id = cm2.user_id
    AND cm1.community_id <> cm2.community_id
WHERE f.friendship_type = 'mutual'
GROUP BY user_id
ORDER BY bridge_ratio DESC
LIMIT 100;

```

Q21. Facebook Marketplace: Calculate median time-to-sell per category.

```

SELECT
    category,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM (sold_at -
listed_at)) / 3600) AS median_hours_to_sell,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM (sold_at -

```

```

listed_at)) / 3600) AS p90_hours,
    COUNT(*) AS sold_listings
FROM marketplace_listings
WHERE sold_at IS NOT NULL
GROUP BY category
ORDER BY median_hours_to_sell;

```

Q22. Reels: Find creators whose reels get high initial views but low completion.

```

SELECT
    r.creator_id,
    AVG(rm.initial_views) AS avg_initial_views,
    AVG(rm.completion_rate) AS avg_completion_rate,
    COUNT(r.reel_id) AS reel_count
FROM reels r
JOIN reel_metrics rm ON r.reel_id = rm.reel_id
WHERE r.published_at >= NOW() - INTERVAL '30 days'
GROUP BY r.creator_id
HAVING AVG(rm.initial_views) > 10000
    AND AVG(rm.completion_rate) < 0.30
ORDER BY avg_initial_views DESC;

```

Q23. Identify the "power users" — top 5% by engagement who drive 50%+ of content.

```

WITH user_engagement AS (
    SELECT
        user_id,
        COUNT(*) AS total_actions,
        NTILE(20) OVER (ORDER BY COUNT(*) DESC) AS percentile_bucket
    FROM user_actions
    GROUP BY user_id
),
total AS (SELECT SUM(total_actions) AS grand_total FROM user_engagement)
SELECT
    percentile_bucket,
    COUNT(user_id) AS users,
    SUM(total_actions) AS actions,
    ROUND(100.0 * SUM(total_actions) / (SELECT grand_total FROM total), 2) AS
pct_of_total
FROM user_engagement
GROUP BY percentile_bucket
ORDER BY percentile_bucket;

```

Q24. Privacy: Find all user data that should be deleted under "right to be forgotten" requests.

```

SELECT
    'users' AS table_name, user_id::text AS record_id
FROM users WHERE user_id = :user_id

UNION ALL

```

```

SELECT 'posts', post_id::text FROM posts WHERE author_id = :user_id

UNION ALL

SELECT 'comments', comment_id::text FROM comments WHERE author_id = :user_id

UNION ALL

SELECT 'messages', message_id::text FROM messages WHERE sender_id = :user_id

UNION ALL

SELECT 'ad_profiles', profile_id::text FROM ad_targeting_profiles WHERE user_id = :user_id

UNION ALL

SELECT 'login_history', login_id::text FROM login_history WHERE user_id = :user_id;

```

Q25. Detect coordinated inauthentic behavior: accounts posting identical content simultaneously.

```

WITH content_fingerprints AS (
    SELECT
        MD5(LOWER(REGEXP_REPLACE(content_text, '\s+', ' ', 'g'))) AS content_hash,
        author_id,
        posted_at
    FROM posts
    WHERE posted_at >= NOW() - INTERVAL '24 hours'
),
coordinated AS (
    SELECT
        content_hash,
        COUNT(DISTINCT author_id) AS account_count,
        ARRAY_AGG(DISTINCT author_id) AS accounts,
        MIN(posted_at) AS first_posted,
        MAX(posted_at) AS last_posted
    FROM content_fingerprints
    GROUP BY content_hash
    HAVING COUNT(DISTINCT author_id) >= 5
        AND MAX(posted_at) - MIN(posted_at) < INTERVAL '10 minutes'
)
SELECT * FROM coordinated ORDER BY account_count DESC;

```

4. Database Design Tasks

Task 1: Design Instagram-Scale Feed System

```

-- Core entities
CREATE TABLE users (
    user_id    BIGSERIAL PRIMARY KEY,
    username   VARCHAR(30) UNIQUE NOT NULL,
    account_type VARCHAR(20) DEFAULT 'personal'
);

CREATE TABLE follows (
    follower_id BIGINT NOT NULL,
    followee_id BIGINT NOT NULL,
    followed_at TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (follower_id, followee_id)
);

-- Media posts
CREATE TABLE posts (
    post_id    BIGSERIAL PRIMARY KEY,
    author_id  BIGINT NOT NULL REFERENCES users(user_id),
    post_type  VARCHAR(20) CHECK (post_type IN
('photo', 'video', 'reel', 'story', 'carousel')),
    caption    TEXT,
    location   POINT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    expires_at TIMESTAMPTZ -- for stories
) PARTITION BY RANGE (created_at);

-- Fan-out on write (push model) for small accounts
-- Fan-out on read for accounts with > 1M followers
CREATE TABLE feed_items (
    user_id    BIGINT NOT NULL,
    post_id    BIGINT NOT NULL,
    score       FLOAT8 NOT NULL,
    inserted_at TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (user_id, post_id)
) PARTITION BY HASH (user_id);

-- Feed index
CREATE INDEX idx_feed_user_score ON feed_items (user_id, score DESC, inserted_at
DESC);

```

Task 2: Design WhatsApp Message Storage at Scale

```

CREATE TABLE conversations (
    conversation_id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    conversation_type VARCHAR(10) CHECK (conversation_type IN ('dm', 'group')),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE conversation_members (
    conversation_id UUID NOT NULL,

```

```

    user_id          BIGINT NOT NULL,
    joined_at        TIMESTAMPTZ DEFAULT NOW(),
    role             VARCHAR(20) DEFAULT 'member',
    PRIMARY KEY (conversation_id, user_id)
);

CREATE TABLE messages (
    message_id       UUID DEFAULT gen_random_uuid(),
    conversation_id   UUID NOT NULL,
    sender_id        BIGINT NOT NULL,
    message_type      VARCHAR(20) CHECK (message_type IN
('text','image','video','audio','document')),
    content           TEXT, -- encrypted at application layer
    sent_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (message_id, sent_at)
) PARTITION BY RANGE (sent_at);

CREATE TABLE message_receipts (
    message_id       UUID NOT NULL,
    recipient_id     BIGINT NOT NULL,
    delivered_at      TIMESTAMPTZ,
    read_at          TIMESTAMPTZ,
    PRIMARY KEY (message_id, recipient_id)
);

-- Efficient unread count
CREATE INDEX idx_receipts_unread ON message_receipts (recipient_id, delivered_at)
WHERE read_at IS NULL;

```

Task 3: Design Ads Targeting and Attribution System

```

CREATE TABLE ad_targeting_segments (
    segment_id       BIGSERIAL PRIMARY KEY,
    segment_name      VARCHAR(200),
    segment_type      VARCHAR(50), -- 'interest','demographic','lookalike','custom'
    created_by        BIGINT,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE user_segment_memberships (
    user_id          BIGINT NOT NULL,
    segment_id       BIGINT NOT NULL,
    score            NUMERIC(6,4) DEFAULT 1.0,
    updated_at       TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (user_id, segment_id)
);

CREATE TABLE ad_campaigns (
    campaign_id      BIGSERIAL PRIMARY KEY,
    advertiser_id     BIGINT NOT NULL,
    objective         VARCHAR(50),

```

```

    daily_budget    NUMERIC(12,2),
    start_date      DATE,
    end_date        DATE,
    status          VARCHAR(20) DEFAULT 'active'
);

CREATE TABLE ad_impressions (
    impression_id   UUID DEFAULT gen_random_uuid(),
    ad_id           BIGINT NOT NULL,
    user_id         BIGINT NOT NULL,
    placement       VARCHAR(50),
    shown_at        TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (impression_id, shown_at)
) PARTITION BY RANGE (shown_at);

CREATE TABLE ad_clicks (
    click_id        UUID PRIMARY KEY,
    impression_id   UUID NOT NULL,
    ad_id           BIGINT NOT NULL,
    user_id         BIGINT NOT NULL,
    clicked_at      TIMESTAMPTZ NOT NULL
);

CREATE TABLE conversions (
    conversion_id   UUID PRIMARY KEY,
    click_id        UUID REFERENCES ad_clicks(click_id),
    ad_id           BIGINT NOT NULL,
    user_id         BIGINT NOT NULL,
    conversion_type VARCHAR(50),
    order_value     NUMERIC(12,2),
    converted_at    TIMESTAMPTZ NOT NULL
);

```

5. System Design with DB Focus

Scenario 1: Design Instagram's Like Counter at Petabyte Scale

Challenge: 1 trillion likes total; 1 billion new likes/day. Display like counts in real-time.

Architecture:

Write Path:

- User likes post → Kafka topic (likes_stream)
 - Flink (deduplication, 5-min tumbling window aggregation)
 - Redis INCR (real-time counter, TTL 24h for hot posts)
 - Cassandra (durable like events, append-only)
 - Batch job: daily rollup to PostgreSQL (for analytics)

Read Path:

- GET /post/{id}/likes →
 - L1: Redis (< 1ms, hot posts)

L2: Cassandra (< 5ms, warm posts)
L3: PostgreSQL (analytics aggregate)

Exact vs. Approximate:

Hot posts: exact Redis counter (strong consistency)
Cold posts: PostgreSQL aggregate (eventual)
Display: "1.2M likes" (formatted, not exact – acceptable for UX)

Scenario 2: Design Facebook's News Feed Ranking

DB Schema decisions for ranking at scale:

```
-- Feature store for ranking model
CREATE TABLE feed_candidate_features (
  user_id          BIGINT NOT NULL,
  post_id          BIGINT NOT NULL,
  feature_vector   FLOAT4[] NOT NULL, -- ML features for ranking model
  computed_at      TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (user_id, post_id)
) PARTITION BY HASH (user_id);

-- Pre-computed affinity scores (social graph strength)
CREATE TABLE user_affinity (
  user_id          BIGINT NOT NULL,
  target_user_id   BIGINT NOT NULL,
  affinity_score    FLOAT4 NOT NULL,
  last_interaction TIMESTAMPTZ,
  PRIMARY KEY (user_id, target_user_id)
);

-- Ranked feed cache (pre-computed, refreshed periodically)
CREATE TABLE ranked_feed_cache (
  user_id          BIGINT NOT NULL,
  post_id          BIGINT NOT NULL,
  rank_position     INT NOT NULL,
  rank_score        FLOAT4 NOT NULL,
  cached_at        TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (user_id, rank_position)
) PARTITION BY HASH (user_id);
```

Scenario 3: Design Real-Time Ads Auction System

Requirements: 10M auctions/second, winner determined in < 10ms.

Architecture:

```
Auction Request → Load Balancer → Auction Service (stateless)
                                   ↓
                               Redis Cluster (budget tracking, frequency caps)
                                   ↓
                               Feature Store (RocksDB) – user targeting features
                                   ↓
```

ML Scoring Service – predict CTR

↓

Winner selection + price calculation

↓

Kafka → PostgreSQL (async: impression log, billing)

PostgreSQL role:

- Billing reconciliation (daily batch)
- Campaign budget enforcement (near real-time with Redis shadow)
- Reporting and analytics (OLAP queries)
- Audit trail (compliance)

6. Behavioral Questions

Scale and Impact

- "Tell me about a time you built something that handled massive data volume. What challenges arose?"
- "Describe a data pipeline you built that other teams depended on. How did you ensure reliability?"
- "Give an example of a database optimization that had measurable user impact."

Move Fast

- "Tell me about a time you shipped a database change under pressure. What corners did you cut? What didn't you cut?"
- "Describe a quick MVP you built that eventually became a production system."

Data Quality

- "How have you ensured data integrity in a high-volume system?"
- "Tell me about a time you discovered a data quality issue in production. How did you handle it?"

7. Mock Interview Script

Problem: "Design a database for Facebook Events — users create events, invite friends, track RSVPs."

Interviewer: "Let's design a database for Facebook Events at scale."

Candidate clarifies:

1. "What's the expected scale — how many events/day, RSVPs/event?"
2. "Does location/geo matter for discovery?"
3. "Are events private, public, or both?"

Schema design:

```
CREATE TABLE events (  
  event_id      BIGSERIAL PRIMARY KEY,  
  creator_id    BIGINT NOT NULL,  
  title         VARCHAR(500) NOT NULL,  
  description    TEXT,  
  privacy       VARCHAR(20) CHECK (privacy IN
```

```

('public','friends','private','invited_only')),
    starts_at      TIMESTAMPTZ NOT NULL,
    ends_at        TIMESTAMPTZ,
    location        GEOMETRY(POINT, 4326),
    venue_name      VARCHAR(500),
    max_attendees   INT,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE event_invitations (
    event_id        BIGINT NOT NULL REFERENCES events(event_id),
    invitee_id      BIGINT NOT NULL,
    invited_by      BIGINT NOT NULL,
    rsvp_status      VARCHAR(20) DEFAULT 'pending'
        CHECK (rsvp_status IN ('pending','going','maybe','declined')),
    invited_at      TIMESTAMPTZ DEFAULT NOW(),
    responded_at    TIMESTAMPTZ,
    PRIMARY KEY (event_id, invitee_id)
);

-- Index for "my upcoming events" page
CREATE INDEX idx_events_starts_at ON events (creator_id, starts_at)
WHERE starts_at >= NOW();

-- GiST for geo discovery
CREATE INDEX idx_events_location ON events USING GIST (location)
WHERE privacy = 'public';

-- Count RSVPs efficiently
CREATE MATERIALIZED VIEW event_rsvp_counts AS
SELECT
    event_id,
    COUNT(*) FILTER (WHERE rsvp_status = 'going') AS going_count,
    COUNT(*) FILTER (WHERE rsvp_status = 'maybe') AS maybe_count,
    COUNT(*) FILTER (WHERE rsvp_status = 'declined') AS declined_count
FROM event_invitations
GROUP BY event_id
WITH DATA;

```

Interviewer: "How do you handle a popular event with 100K RSVPs and everyone updating simultaneously?"

Candidate: "Row-level counters would be a hotspot. I'd use Redis INCR for real-time counts, with the materialized view as a periodic sync. For RSVP writes, I'd use a queue (Kafka) to absorb spikes and batch-update the database. The RSVP `status` update is idempotent (upsert), so retries are safe:

```

INSERT INTO event_invitations (event_id, invitee_id, invited_by, rsvp_status,
responded_at)
VALUES ($1, $2, $3, $4, NOW())
ON CONFLICT (event_id, invitee_id) DO UPDATE
SET rsvp_status = EXCLUDED.rsvp_status, responded_at = NOW();
```

```

---

## ## 8. Evaluation Rubric

### ### E4 (Mid-Level)

|                |                                |                                    |
|----------------|--------------------------------|------------------------------------|
| Signal         | Pass                           | Exceeds                            |
| ---            | ---                            | ---                                |
| SQL            | Window functions, aggregations | Optimizes for billions of rows     |
| Scale Thinking | Mentions indexes               | Discusses denormalization, caching |
| Systems        | Basic design                   | Mentions distributed patterns      |
| Behavioral     | Clear stories                  | Impact on millions of users        |

### ### E5 (Senior)

|                   |                                    |                                      |
|-------------------|------------------------------------|--------------------------------------|
| Signal            | Pass                               | Exceeds                              |
| ---               | ---                                | ---                                  |
| SQL               | Hard queries, correct optimization | Novel approaches, distributed SQL    |
| Data Architecture | Complete designs                   | Discusses Meta's actual patterns     |
| Scale             | Handles billions                   | Petabyte-scale reasoning             |
| Leadership        | Has driven team decisions          | Has influenced org-level data design |

---

## ## 9. Red Flags

- Designs that break at 1M rows (no partitioning, no sharding discussion)
- No mention of caching layer in any design
- Cannot explain fan-out-on-write vs. fan-out-on-read trade-offs
- SQL that works but ignores performance (no thought about indexes)
- Behavioral stories with vague impact ("improved performance")
- No familiarity with distributed SQL (Presto/Trino, BigQuery, Spark SQL)
- Doesn't ask about scale before designing

---

## ## 10. Green Flags

- Immediately asks "what's the scale?" before any design
- Naturally discusses caching, message queues, and async processing
- Knows Meta's tech stack (Presto, TAO, Scribe, Hive)
- SQL uses approximate aggregations for scale (APPROX\_DISTINCT)
- Behavioral stories cite user-facing metrics (DAU impact, latency reduction)
- Discusses privacy/GDPR constraints without prompting
- Shows awareness of hot row / write amplification problems

---

## ## 11. Salary Bands (Approximate, USD, 2024-2025)

|       |               |              |               |               |
|-------|---------------|--------------|---------------|---------------|
| Level | Base          | Annual Bonus | RSU (4yr)     | Total Comp    |
| ---   | ---           | ---          | ---           | ---           |
| E3    | \$150K-\$180K | 10%          | \$80K-\$150K  | \$180K-\$240K |
| E4    | \$180K-\$220K | 15%          | \$150K-\$280K | \$240K-\$370K |
| E5    | \$220K-\$280K | 20%          | \$280K-\$560K | \$390K-\$620K |
| E6    | \$280K-\$360K | 25%          | \$600K-\$1.2M | \$700K-\$1.2M |

| E7 | \$360K-\$430K | 30% | \$1.2M-\$2.5M | \$1.2M-\$2M |

\*Menlo Park/NYC rates. Remote ~10-15% lower.\*

---

## ## 12. Preparation Timeline (4-Week Plan)

### ### Week 1: Scale-First SQL

- Day 1-2: Study Meta's data infrastructure (TAO, Presto, Scribe – public blog posts)
- Day 3: Practice SQL on large-scale scenarios (billions of rows framing)
- Day 4-5: Window functions, approximate aggregations, fan-out patterns
- Day 6-7: A/B testing analysis queries, funnel analysis, cohort retention

### ### Week 2: Distributed Data Design

- Day 1-2: Study distributed databases: Cassandra, HBase data models
- Day 3-4: Design social graph, news feed, messaging systems
- Day 5: Partitioning strategies at petabyte scale
- Day 6-7: Practice mock design interview for Instagram-scale problems

### ### Week 3: Real Systems

- Day 1: Ads auction and attribution system design
- Day 2: Privacy + GDPR patterns in large-scale systems
- Day 3-4: Presto/Trino query optimization (Meta's internal analytics)
- Day 5: Content moderation systems (policy violation detection)
- Day 6-7: Mock behavioral + technical combined rounds

### ### Week 4: Polish

- Day 1-2: Behavioral stories with scale-impact framing
- Day 3: Meta-specific: Instagram, WhatsApp, Marketplace problem scenarios
- Day 4-5: Hard SQL problems from Meta's HackerRank bank
- Day 6-7: Full mock loops + rest